

rockwell_kyle_project5_part2_report

Kyle Rockwell

Analysis of LRU Hash Tables, Single vs Double Probing

Hypothesis

I expect the double probing to be less efficient time wise, with the LRU profile specifically since the LRU profile requires a lot of checking that the next input is in the array, which results in reading the same areas of memory. I think the cache friendlier single probing will

benefit from this. It seems more likely that with single probing if value isn't in the first location, it might still be close enough that it is in the same cache line. Meanwhile with double probing, if a value cant be inserted in the position of the initial hash, its very unlikely that it ends up close enough to be in the same cache line. I think because of the cache inefficiency along with the extra work required to hash each value, that the double probing will cost more time on average.

I think that with smaller N's that the difference between the two will be smaller, but I still think that double probing will be slower. With large N's I think the difference between them will grow and double probing again will be slower

as far as Average Probes, I think that the difference will be small at the smaller N, while at the larger N, double probing will be more expensive.

I think that with the histograms of the two,

Before compaction:

single probing will have longer runs since by nature of single probing if something cant be inserted into the initial location, it will be inserted into the next available spot.

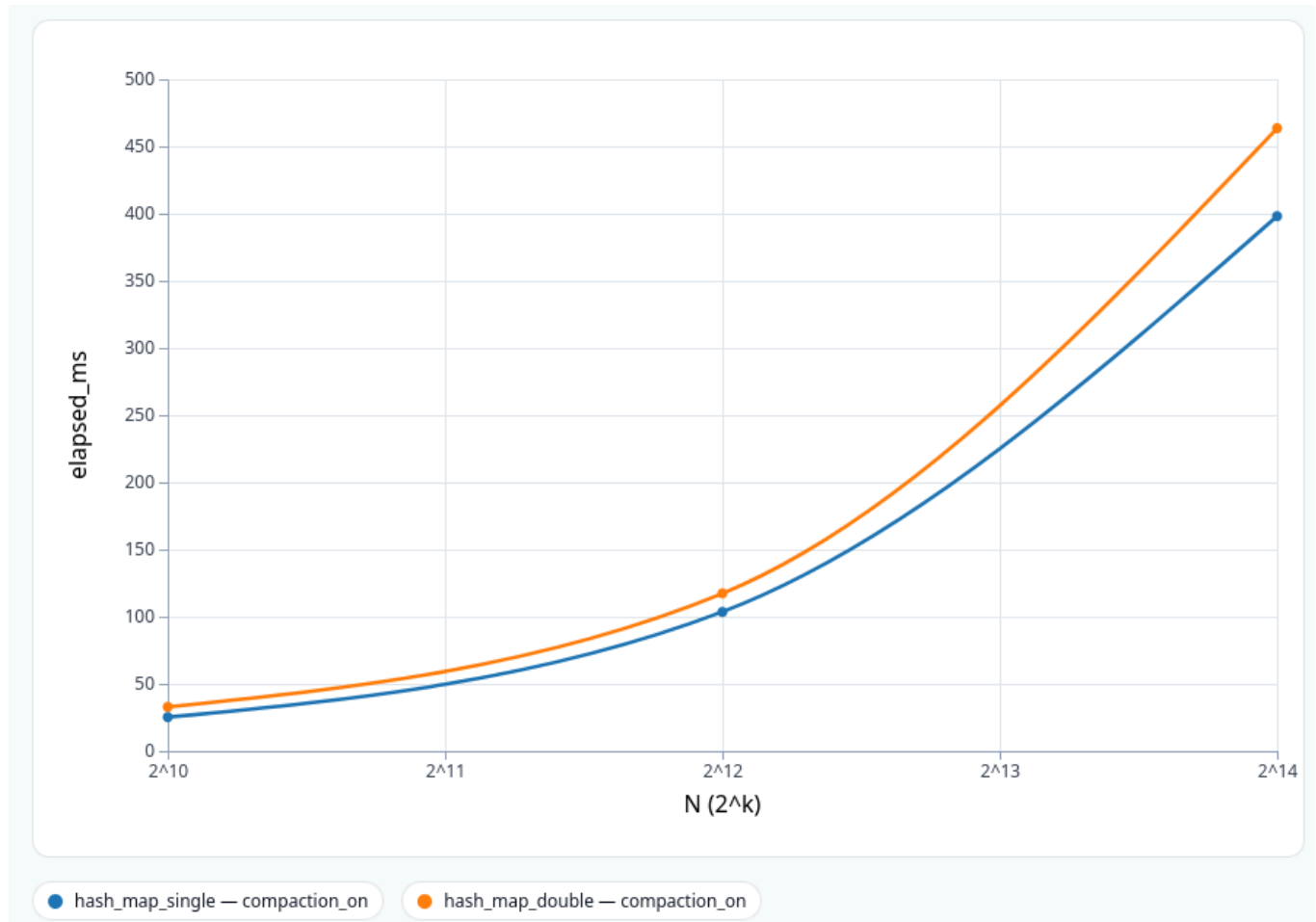
After compaction:

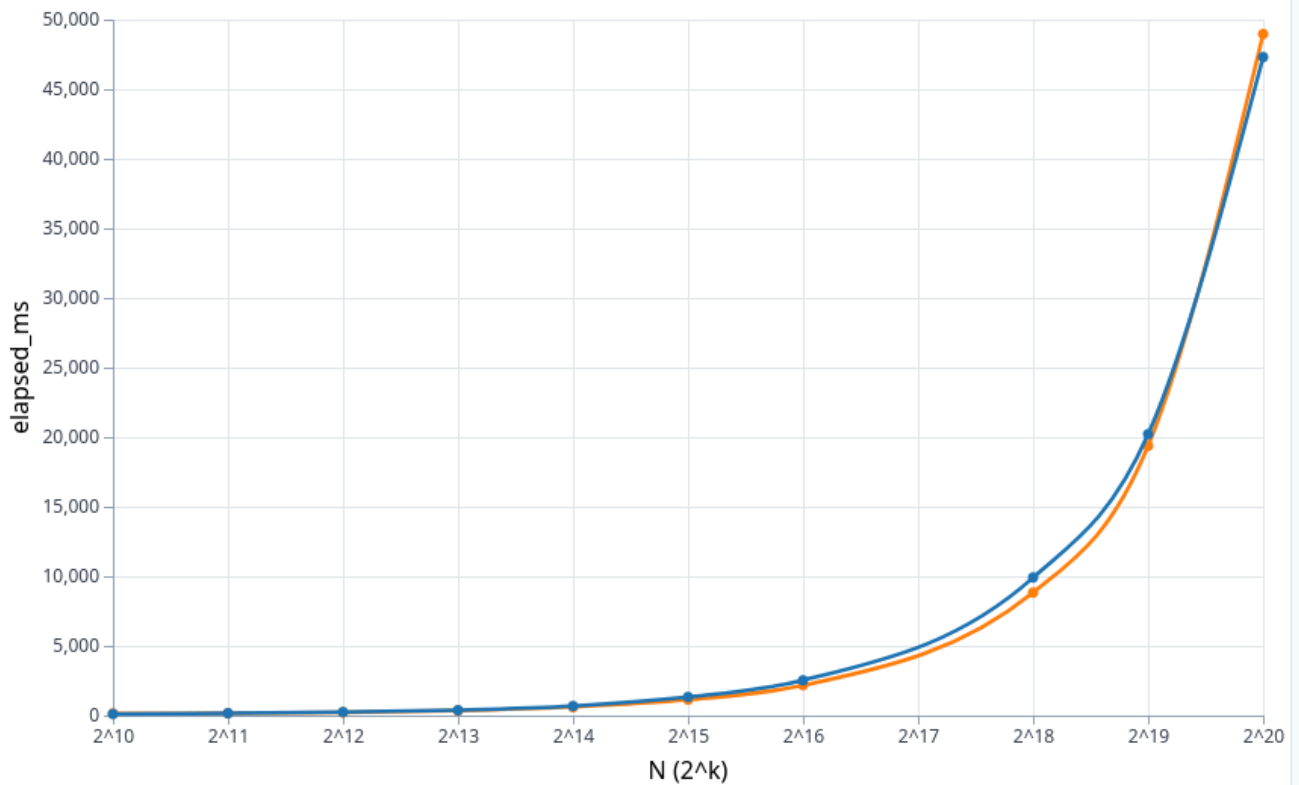
I think that single probing will have most runs in the 1-5 length, since the data in the traces is either repeated 1, 3 or 5 times. I think the double probing will vary more in run length, and I dont have any reason but my intuition tells me that double will have longer runs on average after compaction.

Results

"After plotting elapsed time vs. N, where does the data agree or disagree with your expectation? Identify any N-range where the gap widens or narrows notably."

The first graph below is a timing of the trace files where $n = 4096$ and 65536 while the second graph is a timing of all the trace files.





● hash_map_double — compaction_on

● hash_map_single — compaction_on

as we can see the first graph follows my hypothesis pretty closely, while the second graph starts with single and double probing being roughly the same time, then single probing becomes slower, with double probing taking longer at 2^{20} .

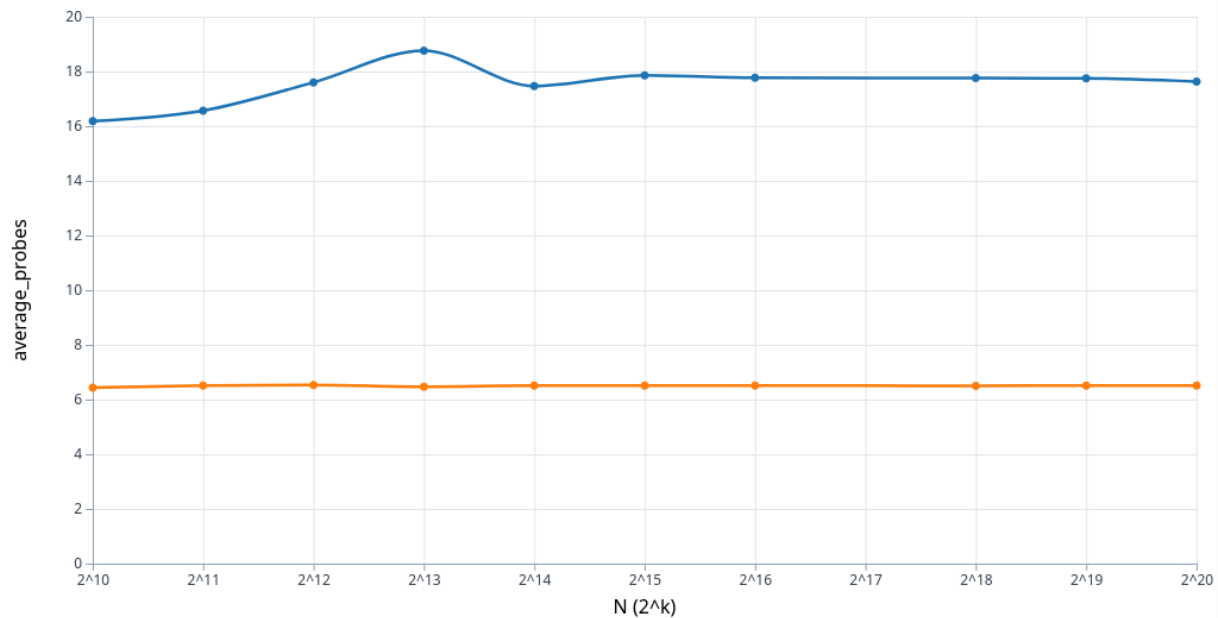
The difference between single probing and double probing at this value is 3.4% which might be due to a difference in efficiency single vs double probing, or within a margin of error.

The fact that the two graphs tell different stories suggests to me that more testing should be done.

work per operation vs wall-clock time

Compare average_probes vs. elapsed time across N for both methods. Do fewer probes consistently correlate with lower time in your runs?

Below is a graph of the average probes for all the trace files



5.1 Expectations vs. observations

As you can see by the graph my hypothesis was way wrong. The average number of probes didn't seem to change much at all and remained relatively constant. Additionally single probing had almost triple the average probes. This makes sense since with double probing there should be relatively fewer probes needed to insert an element in the hash. While with single probing since elements are inserted into the next available space, meaning the number of probes would go up linearly with the length of the runs.

5.1.2 “Work” per operation vs. wall-clock time

Since the average probes are relatively constant it is hard to correlate this with the elapsed time. It's possible that the compaction caused the average probes to never grow that big. I would like to get the program to run with compaction off, as referenced in the README, in order to see how the number of probes react.

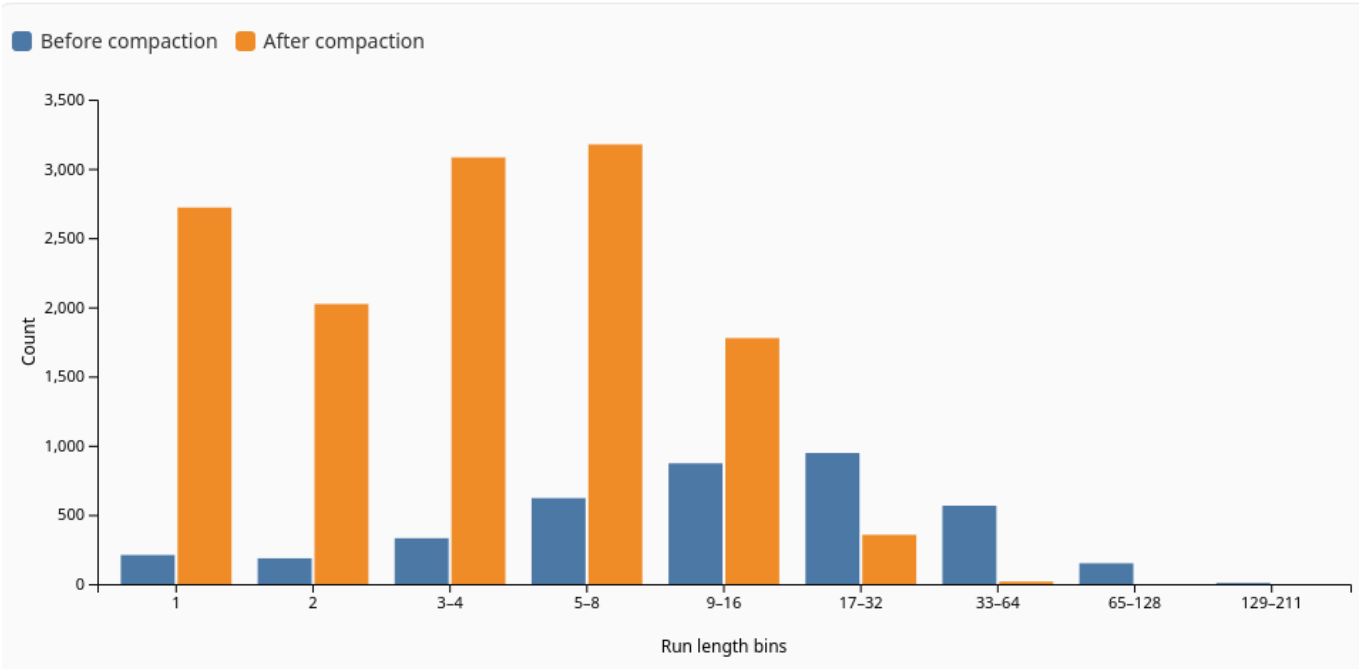
**5.2.4 Histogram Questions to answer

with single probing before compaction there much more elements that were in a long run, and after compaction there were mostly elements in short to medium runs, with a substantial spike at one length runs. This makes sense since after compaction all the dead elements are removed and only the active elements are inserted. Based on the trace inputs the values have between one and 5 copies, so it makes sense that most of the runs are in the 1-5 run length size. There are runs in the higher ranges too, since the data is randomized, so there are still conflicts when it comes to inserting.

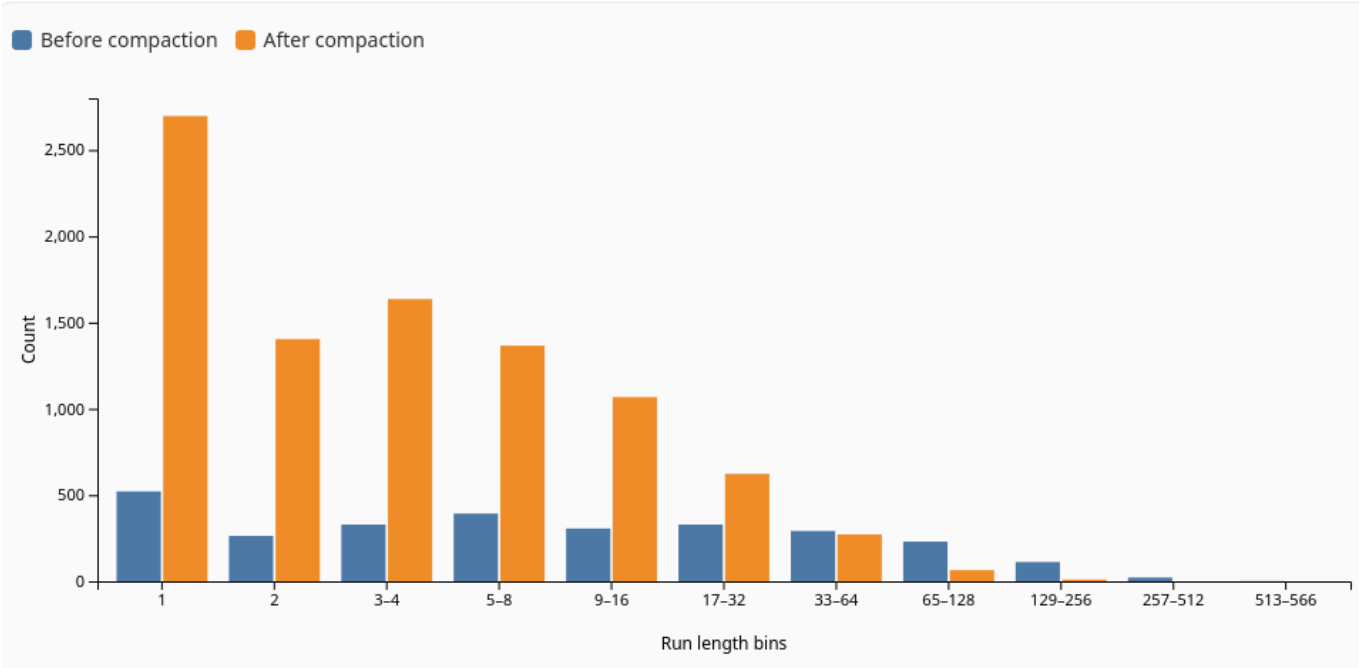
with double probing the story is a little different. Before compaction there are a lot more medium length runs, and the length of the highest run is smaller than the single probing's highest run

(for n = 65536: 211 vs 566). Which is pretty substantial. After compaction, double probing has less of a spike at the one run length, but its mean is smaller, (4.98 vs 7.17) this shows that the double probing results in more shorter length runs after compaction. Which makes sense given the nature of how double probing works, (i.e. not sequentially inserting when there is a conflict)

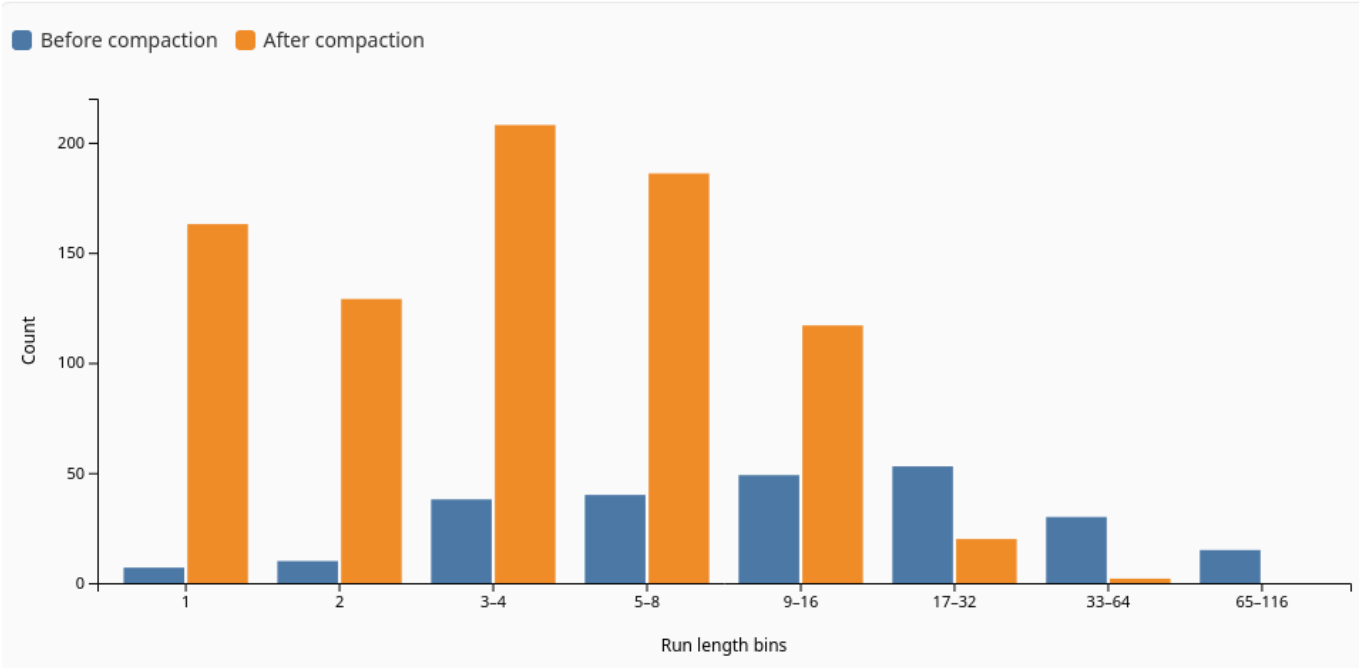
65536 Double Probing



65536 Single Probing



4096 Double Probing



4096 Single Probing

